



PEC3 - Tercera prueba de evaluación continuada

Presentación

Esta PEC se focaliza en los circuitos secuenciales. Los circuitos combinacionales nos permiten describir funcionalidades de un circuito pero no nos permiten guardar información. Mediante biestables y registros podemos guardar información en memoria y hacer circuitos más complejos. En esta PEC practicaremos con este tipo de circuitos.

Competencias

- Entender el funcionamiento de los circuitos lógicos secuenciales, y conocer y saber aplicar técnicas de diseño de sistemas secuenciales.

Objetivos

- Saber discernir, a partir de la funcionalidad que se quiere que tenga un circuito lógico, si el circuito tiene que ser de tipo secuencial o combinacional.
- Conocer el funcionamiento del biestable D y de todas las entradas de control que puede tener.
- Saber analizar un circuito secuencial.
- Saber realizar un cronograma a partir d'un circuito digital secuencial.
- Saber analizar un grafo de estados.
- Saber diseñar un circuito cualquiera a partir de la descripción de su funcionalidad mediante el modelo de Moore.

Recursos

Los recursos que se recomienda usar para esta PEC son los siguientes:

Básicos: El módulo 4 de los materiales.

Complementarios: VerilCIRC, VerilCHART y el Wiki de la asignatura.

Criterios de valoración

- Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.
- La valoración esta indicada en cada uno de los subapartados.



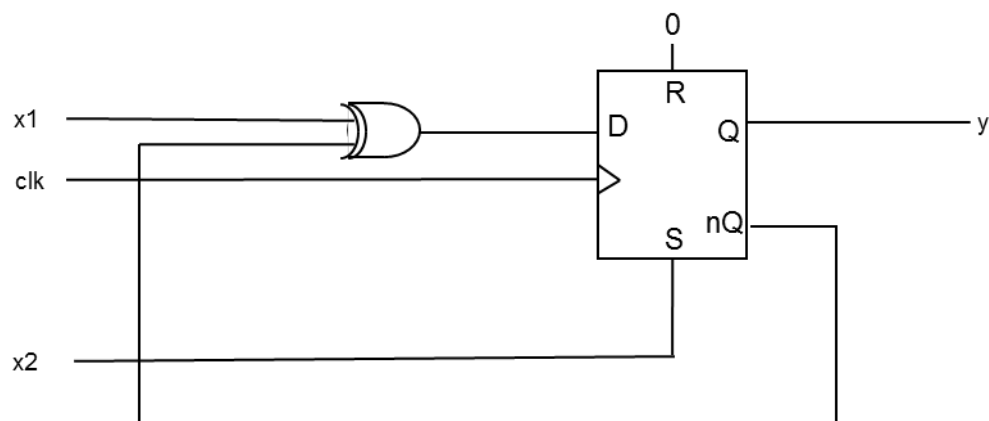
Formato y fecha de entrega

- Para dudas o aclaraciones sobre el enunciado, dirigiós al consultor responsable de vuestra aula.
- Hay que entregar la solución en un fichero PDF usando una de las plantillas proporcionadas conjuntamente con este enunciado.
- Se tiene que entregar a través de la aplicación de **Entrega y registro de EC** del apartado Evaluación de vuestra aula.
- La fecha límite de entrega es el **28 de abril** (a las 24 horas).

Solución de la PEC

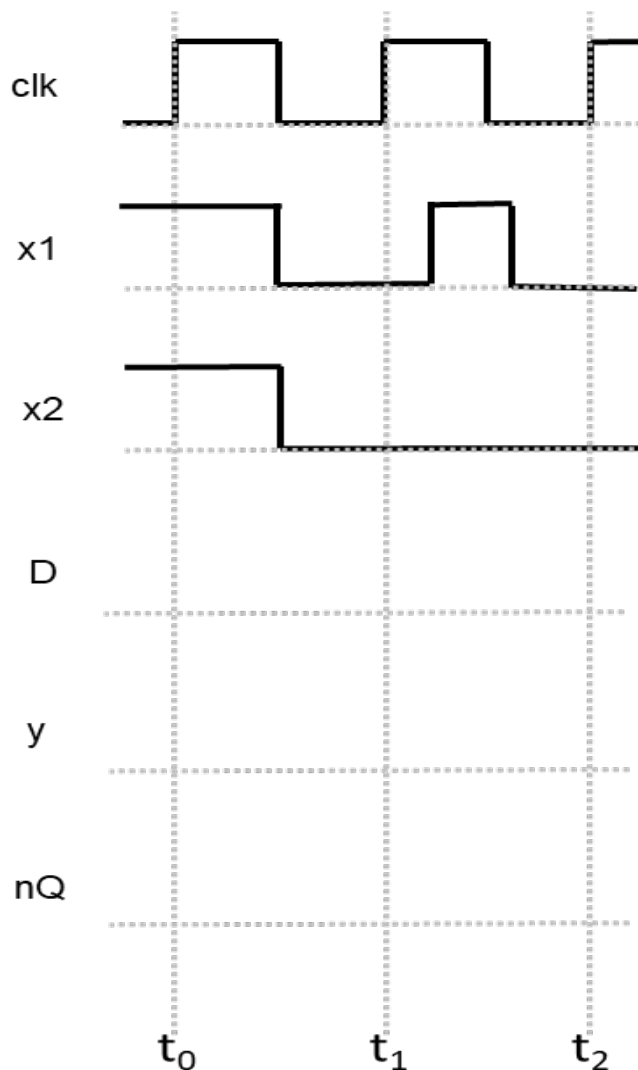
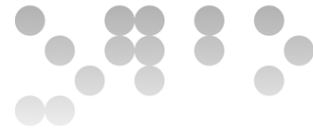
Ejercicio 1 (15%)

Dado el circuito que se muestra a continuación:



Completad el siguiente cronograma e indicad **razonadamente** el valor que toma cada señal (D , y nQ) en el **momento inicial** y en **cada flanco ascendente** de la señal de reloj (instantes t_0 , t_1 i t_2).

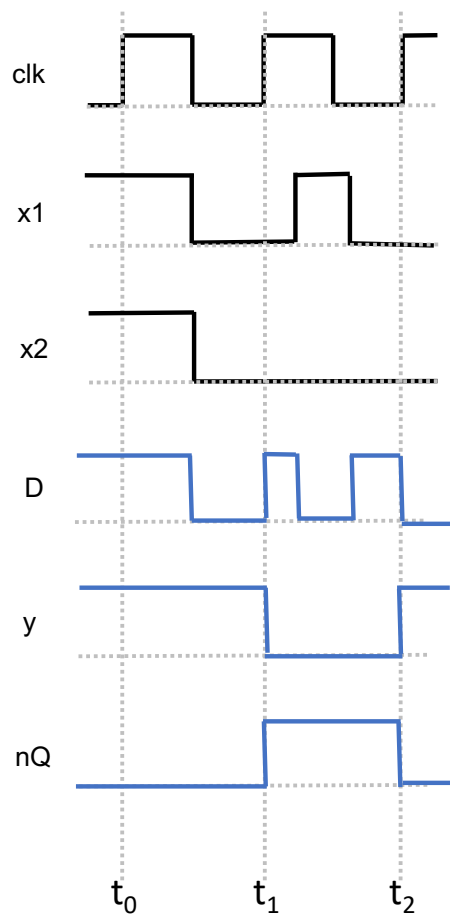
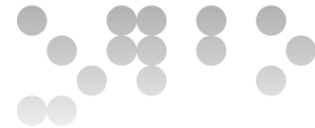
Nota: Tenéis disponible este ejercicio en VeriIChart.



Señal D : corresponde al valor de la salida de una puerta XOR con las entradas $x1$ y nQ . El valor de D aplica la tabla de verdad de la puerta XOR: la salida de la puerta es 1 cuando las dos entradas tienen un valor diferente, y 0 cuando las entradas tienen valor igual. El valor de salida se produce de forma inmediata cuando hay un cambio en la entrada.

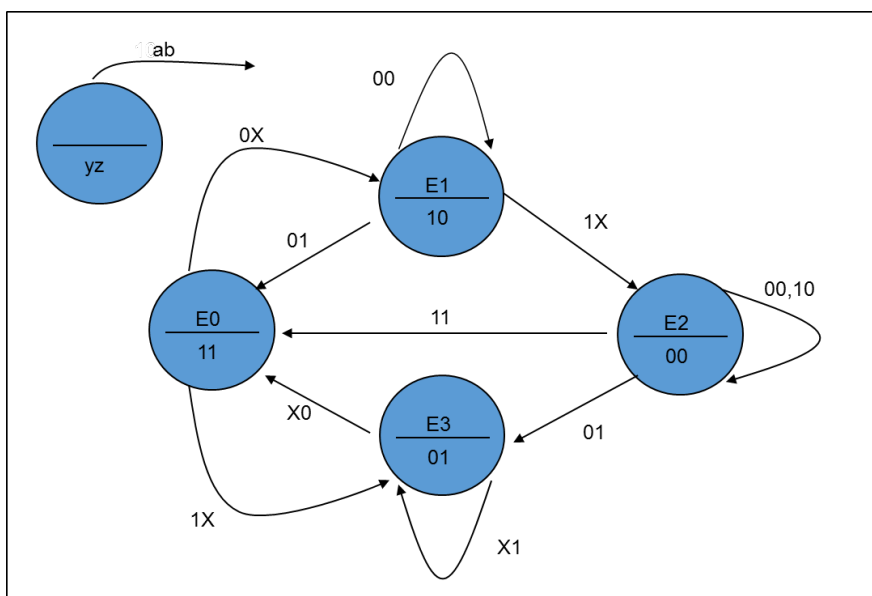
Señal y : corresponde a la salida Q del biestable. El valor $x2=1$ activa la entrada *set* del biestable y hace que tenemos $y=1$ hasta el instante t_1 . La entrada *set* es una entrada asíncrona y tiene prioridad mayor sobre la entrada D . Con $x2=1$ en t_0 el valor de D en el instante t_0 es indiferente. En el instante t_1 , $x2=0$ el biestable toma el valor que hay en la entrada D (que es 0). Por lo tanto, entre t_1 y t_2 tenemos $y=0$. En el instante t_2 tenemos $D=1$ e y toma el valor 1.

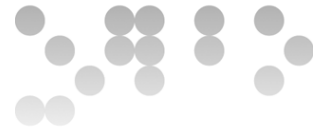
Señal nQ : es la negación de la señal Q que en el circuito corresponde a la salida y .



Ejercicio 2 (40%)

Dado el grafo de estados siguiente:





a) [10%] Escribid la tabla de transiciones y la tabla de salidas del sistema representado por el grafo.

Nota: Tenéis disponible el ejercicio en VerilCHART. Fijaos que la tabla de transiciones y la de salidas se han unificado en una única tabla de excitaciones.

La tabla de transiciones establece cuál es el estado en el ciclo siguiente a partir del valor que toman las señales de entrada.

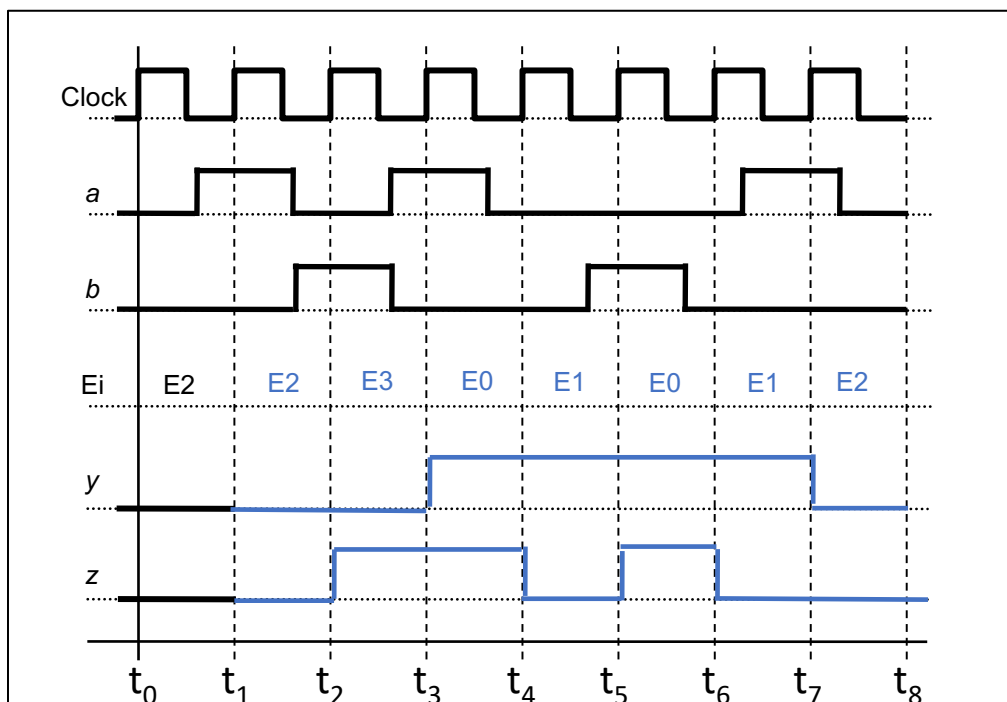
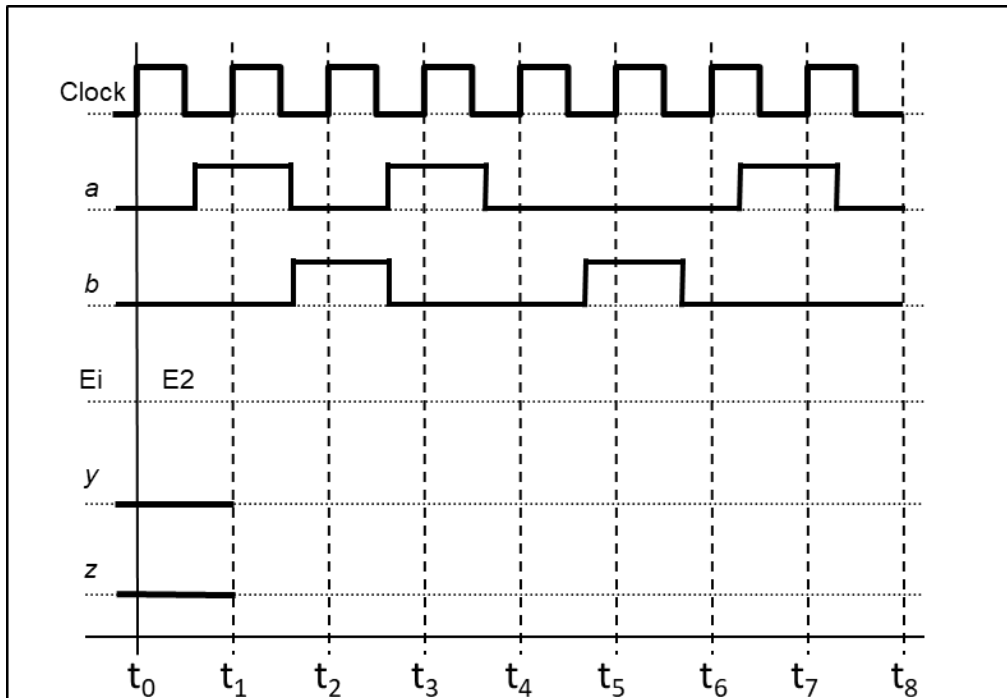
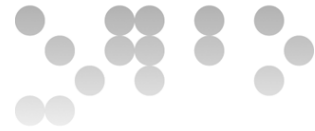
Estado	a	b	Estado+
E0	0	0	E1
E0	0	1	E1
E0	1	0	E3
E0	1	1	E3
E1	0	0	E1
E1	0	1	E0
E1	1	0	E2
E1	1	1	E2
E2	0	0	E2
E2	0	1	E3
E2	1	0	E2
E2	1	1	E0
E3	0	0	E0
E3	0	1	E3
E3	1	0	E0
E3	1	1	E3

La tabla de salidas determina el valor de las señales de salida para cada estado.

Estado	y z
E0	11
E1	10
E2	00
E3	01

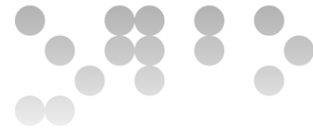
b) [15%] Completad el cronograma siguiente con el valor que toman las señales y y z, y el nombre del estado en cada ciclo.

Nota: Tenéis disponible este ejercicio en VerilChart.



c) [5%] Se desea implementar el circuito representado por el grafo utilizando una memoria ROM y biestables.

¿Cuántos bits son necesarios para la entrada de direcciones de la ROM? ¿Qué tamaño (en palabras) debe tener la memoria ROM? ¿Cuál es el número de bits que debe tener cada palabra de la memoria ROM?



El circuito tiene dos bits de entrada, denominados a y b . Los bits de salida son dos, denominados y y z . Como el circuito tiene 4 estados necesitamos 2 bits para representarlos y, por lo tanto, necesitamos 2 biestables para poder almacenar estos 2 bits.

La memoria ROM que puede implementar este circuito tendría 4 bits de direccionamiento, 2 bits para el estado más 2 bits para la entrada. El ancho de las palabras de la ROM tendría que ser de 4 bits, 2 bits para guardar la salida y 2 bits para codificar el estado futuro.

d) [10%] Implementad el circuito representado por el grafo utilizando una memoria ROM, biestables D y aplicando la codificación de estados siguiente:

Estado	Q_1	Q_0
E0	1	1
E1	1	0
E2	0	0
E3	0	1

Especificad claramente el orden de las señales tanto de direccionamiento como de datos de la memoria ROM. Indicad el contenido de la memoria ROM.

Nota: Tenéis disponible el ejercicio en VerilCIRC.

Hacemos la tabla de transiciones aplicando la codificación de los estados dada.

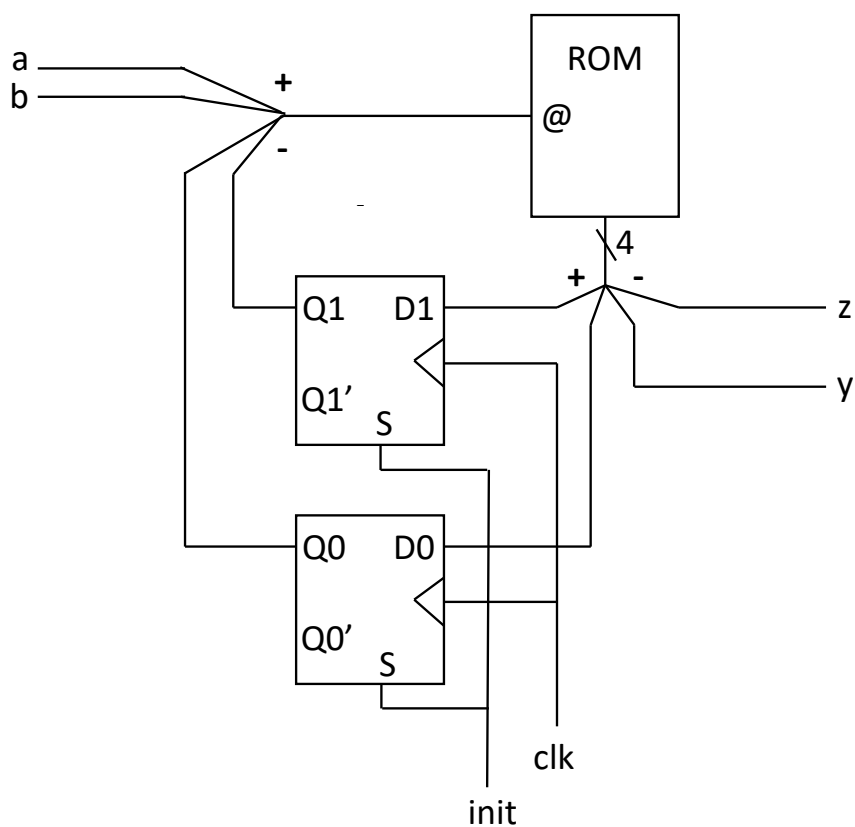
Q_1 Q_0	a b	Q_1+ Q_0+
00	00	00
00	01	01
00	10	00
00	11	11
01	00	11
01	01	01
01	10	11
01	11	01
10	00	10
10	01	11
10	10	00
10	11	00
11	00	10
11	01	10
11	10	01
11	11	01

Unificamos la tabla de transiciones y la tabla de salidas en una, teniendo en cuenta que las salidas (y , z) dependen del estado, no del estado+. A continuación, se muestra la tabla unificada:



$Q_1 Q_0$	a b	$Q_1+ Q_0+$	y z
00	00	00	00
00	01	01	00
00	10	00	00
00	11	11	00
01	00	11	01
01	01	01	01
01	10	11	01
01	11	01	01
10	00	10	10
10	01	11	10
10	10	00	10
10	11	00	10
11	00	10	11
11	01	10	11
11	10	01	11
11	11	01	11

La implementación de la tabla se realiza con el siguiente circuito. En este circuito la ROM de 16x4 bits contiene el cálculo del estado siguiente y las salidas.





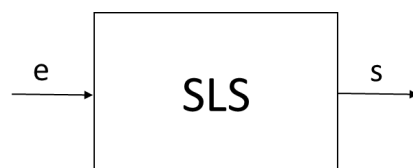
El contenido de la memoria ROM corresponde con la tabla unificada de transiciones y salidas:

@	$d_1 d_0$ y z
0000	0000
0001	0100
0010	0000
0011	1100
0100	1101
0101	0101
0110	1101
0111	0101
1000	1010
1001	1110
1010	0010
1011	0010
1100	1011
1101	1011
1110	0111
1111	0111

Dado que el valor de las salidas y el valor del estado coinciden, existe otra solución más eficiente consistente en usar una ROM de 16x2 bits para el cálculo del estado siguiente y obtener las salidas y y z a partir de los estados $Q1$ y $Q0$, respectivamente.

Ejercicio 3 (20%)

Diseñad el grafo de estados de un circuito SLS de un bit de entrada y un bit de salida que se comporte de la manera que se describe a continuación.



El funcionamiento es el siguiente: a partir de la inicialización se coge el valor de la entrada en 3 ciclos consecutivos y después pasa a mostrar la suma de estos 3 bits por la salida en dos ciclos consecutivos, el bit de más peso primero y el de menos peso después. Este mismo proceso se repite cada 5 ciclos. El valor de la salida durante los 3 primeros ciclos de cada secuencia vale 0.

Ejemplo:

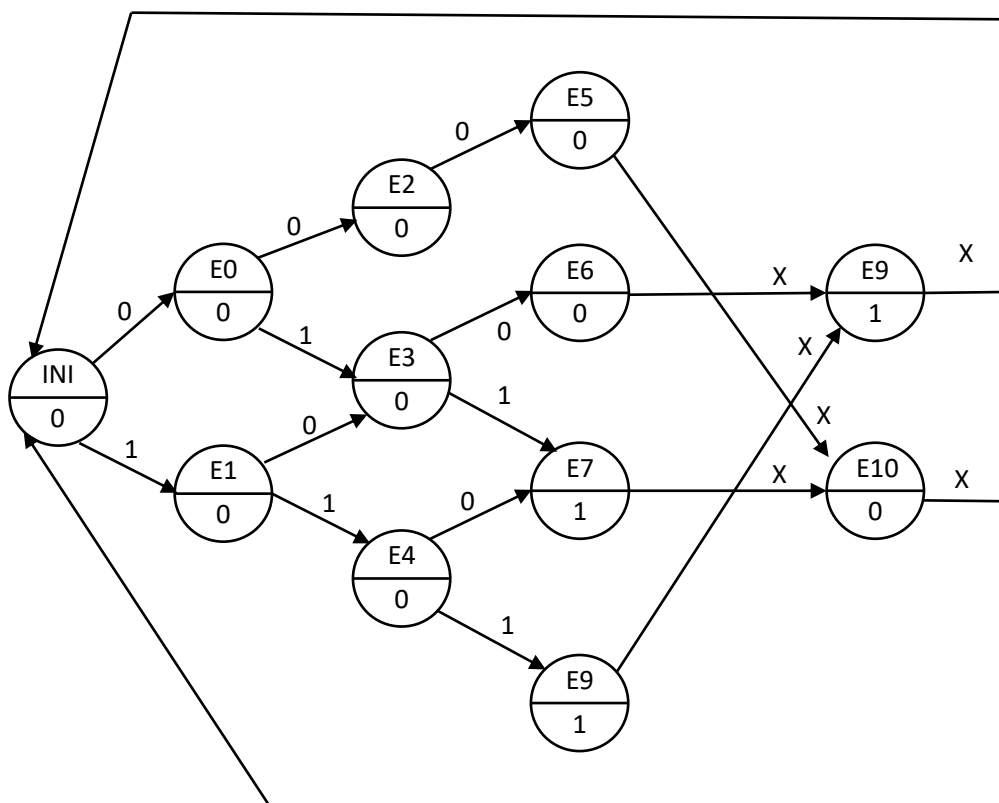
- Secuencia de entrada: 010xx111xx...
- Secuencia de salida: 0000100011...

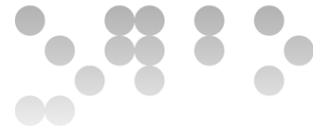


Para conseguir el funcionamiento deseado, el circuito tiene que tener los siguientes estados:

Estado	Descripción	Salida
INI	Estado inicial, no se ha reconocido nada de la secuencia	0
E0	Se ha visto un 0	0
E1	Se ha visto un 1	0
E2	Se ha visto la secuencia 00	0
E3	Se ha visto la secuencia 01 o 10	0
E4	Se ha visto la secuencia 11	0
E5	Se ha visto la suma 0 (bit de más peso)	0
E6	Se ha visto la suma 1 (bit de más peso)	0
E7	Se ha visto la suma 2 (bit de más peso)	1
E8	Se ha visto la suma 3 (bit de más peso)	1
E9	Se ha visto la suma 1 o la suma 3 (bit de menos peso)	1
E10	Se ha visto la suma 0 o la suma 2 (bit de menos peso)	0

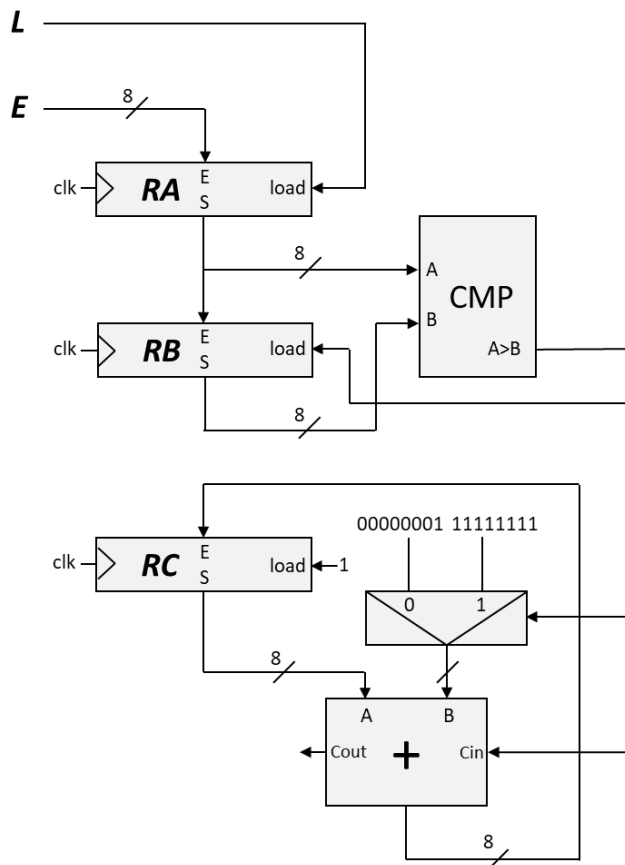
El siguiente grafo representa el comportamiento deseado:





Ejercicio 4 (25%)

Dado el siguiente circuito



Se pide que contestéis los siguientes apartados.

- a) [5%] Indicad **razonadamente** un valor, en el momento de un flanco ascendente del reloj, para cada señal de entrada del circuito (L , E) y para cada registro que contiene (RA , RB y RC) que provoquen que el nuevo valor del registro RA después de este flanco sea 0xAB.

El registro RA depende del valor de las señales L y E . Con $L = 1$ y $E = 0xAB$ se escribirá el valor 0xAB en el registro RA . Los valores de los registros RA , RB y RC en el momento del flanco ascendente son indiferentes.

- b) [10%] Indicad **razonadamente** un valor, en el momento de un flanco ascendente del reloj, para cada señal de entrada del circuito (L , E) y para cada registro que contiene (RA , RB y RC) que provoquen que el nuevo valor del registro RB después de este flanco sea 0xCD.

El registro RB depende del valor de la señal $A > B$ del comparador, que activa la entrada *load*, y del valor del registro RA . Para activar la escritura (entrada *load*=1) del



registro RA , la señal $A > B$ tiene que ser 1 y por tanto el valor de RA tiene que ser mayor que el de RB .

Por lo tanto:

$RA = 0xCD$

RB debe tener un valor menor que RA .

El valor de L , E y RC es indiferente.

- c) [10%] Indicad **razonadamente** un valor, en el momento de un flanco ascendente del reloj, para cada señal de entrada del circuito (L , E) y para cada registro que contiene (RA , RB y RC) que provoquen que el nuevo valor del registro RC después de este flanco sea $0xEF$.

El registro RC carga en cada flanco ascendente el valor de la salida del sumador. Esta salida depende de la salida del multiplexor y de Cin . La salida del multiplexor depende de la señal $A > B$, que depende de los registros RA y RB . Para tener el valor $0xEF$ en el registro RC después del flanco ascendente, en el ciclo anterior RC debe tener el valor $0xEE$. Con $A > B = 0$ el sumador sumaría un 1 que resulta en el valor $0xEF$ en el registro RC después del flanco ascendente. Con $A > B = 1$, dado que $11111111+1$ da la suma 0 en 8 bits, se sumaría un 0.

Por lo tanto:

Cuando tenemos $RC = 0xEE$ en el ciclo anterior y sumar 1: Para tener $A > B = 0$, RA debe tener un valor menor o igual que RB . El valor de L y E es indiferente.

Cuando tenemos $RC = 0xEF$ en el ciclo anterior y sumar 0: Para tener $A > B = 1$, RA tiene que tener un valor mayor que RB . El valor de L y E es indiferente.